

MARAN PMS — Prompt 49: Grupos — Edición Segura de Reserva + Check-out con Pago Total

Archivos involucrados

- `client/src/pages/reservations.tsx` — `ReservationFormDialog` (componente `ReservationEditForm` o similar)
 - `client/src/pages/group-detail.tsx` — `GroupDetailPage`, `dialog de pago grupal`, `checkoutAllMutation`
 - `server/routes.ts` — `POST /api/groups/:id/payment` y `POST /api/groups/:id/checkout-all`
 - `server/storage.ts` — `bulkCheckOut`
-

BUG 1 — Edición de reserva grupal borra datos (CRÍTICO)

Causa raíz

Cuando el usuario navega desde **Grupos** → **Ver detalle** → **tab Reservas** → **fila de reserva**, llega a `ReservationsPage` con el parámetro `?view=id`. El `useEffect` abre `ReservationDetailDialog`, y desde ahí el botón "Editar Reserva" abre `ReservationFormDialog`.

El formulario de edición tiene un **selector de habitaciones** (`availableRooms`) que filtra:

```
const availableRooms = rooms.filter(r =>
  (r.status === "available" || r.id === reservation?.roomId) &&
  r.roomTypeId === selectedRoomTypeId
);
```

El problema: las reservas grupales tienen habitaciones con `status === "occupied"` (la misma reserva las ocupa). El filtro incluye `r.id === reservation?.roomId` para contemplar esto. **Sin embargo**, si `selectedRoomTypeId` se inicializa en vacío `""` (porque el `useEffect` de reset corre antes de que `reservation` esté completamente cargado), el filtro no matchea ninguna habitación. El selector de room queda vacío y al guardar se

envía `roomId: ""`.

El backend tiene:

```
const finalRoomId = req.body.roomId || existing.roomId;
```

Esto protege el `roomId`. **Pero el problema real** está en `selectedRoomTypeId`: cuando queda vacío, el `setFormData` dentro del `useEffect` también puede resetear `roomTypeId` a `""`, y eso sí se envía en el PATCH sobrescribiendo el campo en DB.

Fix 1 — `ReservationFormDialog`: inicialización robusta

En el `useEffect` que resetea el form al abrir, asegurarse de que `selectedRoomTypeId` se setee **antes** que cualquier renderizado del selector:

```
useEffect(() => {
  if (open) {
    // Setear roomTypeId PRIMERO, antes del resto del form
    const roomTypeId = reservation?.roomTypeId || defaultValues?.roomTypeId || ""
    setSelectedRoomTypeId(roomTypeId);

    setSelectedGuest(reservation?.guest || null);
    setSelectedCompany(reservation?.company || null);
    setSelectedAgency(reservation?.agency || null);

    setFormData({
      reservationCode: reservation?.reservationCode || "",
      guestId: reservation?.guestId || "",
      companyId: reservation?.companyId || "",
      agencyId: reservation?.agencyId || "",
      roomTypeId: roomTypeId, // usar la variable ya seteada
      roomId: reservation?.roomId || defaultValues?.roomId || "",
      // ... resto de campos igual que antes
    });
  }
}, [open, reservation]); // IMPORTANTE: quitar "today" y "tomorrow" de las deps
                          // para que no re-ejecute por cambio de fecha
```

Fix 2 — `handleSubmit`: nunca enviar `roomId` o `roomTypeId` vacíos

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  mutation.mutate({
    ...formData,
```

```

    // Protección explícita: si el campo quedó vacío, usar el valor original de l
    roomId: formData.roomId || reservation?.roomId || "",
    roomTypeId: formData.roomTypeId || reservation?.roomTypeId || "",
    guestId: formData.guestId || reservation?.guestId || "",
    bedTypeId: formData.bedTypeId || null,
    nights: Number(formData.nights),
    numberOfGuests: Number(formData.numberOfGuests),
    baseRatePerNight: String(formData.baseRatePerNight || "0"),
    finalRatePerNight: String(formData.finalRatePerNight || "0"),
    totalRoomAmount: String(formData.totalRoomAmount || "0"),
    discountValue: String(formData.discountValue || "0"),
  });
};

```

Fix 3 — Backend: protección adicional en `PATCH /api/reservations/:id`

En `server/routes.ts`, antes de llamar a `storage.updateReservation`, agregar validación explícita:

```

// Si viene roomId vacío, ignorarlo (no actualizar)
if (req.body.roomId === "" || req.body.roomId === null) {
  delete req.body.roomId;
}
// Si viene roomTypeId vacío, ignorarlo
if (req.body.roomTypeId === "" || req.body.roomTypeId === null) {
  delete req.body.roomTypeId;
}
// Si viene guestId vacío, ignorarlo
if (req.body.guestId === "" || req.body.guestId === null) {
  delete req.body.guestId;
}

```

Esto garantiza que incluso si el frontend envía un campo vacío accidentalmente, el backend no sobrescribe el valor existente en DB.

Fix 4 — Auditoría visible de cambios en reserva grupal

El sistema ya registra cambios en `reservation_changelog`. Lo que falta es **mostrarlos en contexto grupal**.

En `GroupDetailPage` (`client/src/pages/group-detail.tsx`), en el tab “Reservas”, agregar en cada fila un indicador de última modificación si existe en el changelog. Opcionalmente, al hacer clic en la fila (que ya navega a `/reservations?view=id`), el `ReservationDetailDialog` ya tiene un tab “Historial” que muestra el changelog — verificar que funcione correctamente para reservas de grupo.

BUG 2 — Check-out grupal bloqueado por saldo (CRÍTICO)

Comportamiento actual

POST /api/groups/:id/check-out-all llama a bulkCheckOut en storage.ts . Si alguna habitación tiene balance > 0.01 , la salta y la agrega a pendingBalance . En grupos con pago centralizado, esto bloquea el cierre aunque el grupo esté pagado en conjunto.

Solución: opción “Con este pago se cierran todas las habitaciones”

Cambio 1 — Frontend: agregar checkbox al dialog de pago grupal

En GroupDetailPage , en el dialog de showGroupPaymentDialog , agregar nuevo estado y UI:

```
const [groupPaymentCloseAll, setGroupPaymentCloseAll] = useState(false);
```

En el body del dialog, agregar después del campo de Referencia:

```
{/* Separador */}
<div className="border-t pt-3">
  <div className="flex items-start gap-3 p-3 rounded-lg border border-amber-200 b
    <Checkbox
      id="close-all-rooms"
      checked={groupPaymentCloseAll}
      onChange={ (v) => setGroupPaymentCloseAll(!v) }
      data-testid="checkbox-close-all-rooms"
    />
  <div className="space-y-1">
    <label
      htmlFor="close-all-rooms"
      className="text-sm font-medium cursor-pointer"
    >
      Con este pago se cierran todas las habitaciones del grupo
    </label>
    <p className="text-xs text-muted-foreground">
      El sistema distribuirá el pago para saldar cada reserva y luego realizará
      el check-out de todas las habitaciones activas. Si hay diferencia con el
      total real, se avisará antes de confirmar.
    </p>
  </div>
</div>
```

```
</div>
</div>
```

Pasar el flag en la mutation:

```
const groupPaymentMutation = useMutation({
  mutationFn: async () => {
    return apiRequest("POST", `/api/groups/${groupId}/payment`, {
      amount: groupPaymentAmount,
      method: groupPaymentMethod,
      reference: groupPaymentReference,
      receiptType: groupPaymentReceiptType,
      distribution: groupPaymentDistribution,
      closeAllRooms: groupPaymentCloseAll, // ← nuevo
    });
  },
  onSuccess: async (res) => {
    const data = await res.json().catch(() => ({}));
    queryClient.invalidateQueries({ queryKey: ["/api/groups", groupId] });
    queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });
    queryClient.invalidateQueries({ predicate: q =>
      Array.isArray(q.queryKey) && q.queryKey[0] === "/api/planning"
    });

    if (groupPaymentCloseAll) {
      if (data.balanceDiff && Math.abs(data.balanceDiff) > 0.01) {
        toast({
          title: "Pago registrado con diferencia",
          description: `Diferencia de ${Math.abs(data.balanceDiff).toFixed(2)} $`,
          variant: "destructive",
        });
      } else {
        toast({
          title: "Pago grupal registrado – grupo cerrado",
          description: `${data.checkoutCount} habitaciones cerradas exitosamente.`,
        });
      }
    } else {
      toast({ title: "Pago grupal registrado exitosamente" });
    }

    setShowGroupPaymentDialog(false);
    setGroupPaymentAmount("");
    setGroupPaymentMethod("");
    setGroupPaymentReference("");
    setGroupPaymentReceiptType("");
  },
});
```

```

    setGroupPaymentDistribution("equal");
    setGroupPaymentCloseAll(false);
  },
  onError: () => {
    toast({ title: "Error al registrar pago grupal", variant: "destructive" });
  },
});

```

Cambio 2 — Backend: POST /api/groups/:id/payment con closeAllRooms

En `server/routes.ts`, extender el endpoint existente:

```

app.post("/api/groups/:groupId/payment", async (req, res) => {
  try {
    const group = await storage.getGroup(req.params.groupId);
    if (!group) return res.status(404).json({ error: "Group not found" });

    const { amount, method, reference, receiptType, distribution, closeAllRooms } = req.body;
    if (!amount || !method) {
      return res.status(400).json({ error: "amount and method are required" });
    }

    const totalAmount = parseFloat(amount);
    if (totalAmount <= 0) {
      return res.status(400).json({ error: "Amount must be positive" });
    }

    const activeReservations = group.reservations.filter(
      r => r.status === "confirmed" || r.status === "checked_in"
    );

    if (activeReservations.length === 0) {
      return res.status(400).json({
        error: "No hay reservas activas para registrar pagos"
      });
    }

    // Si closeAllRooms: calcular el total real del grupo para informar diferencia
    let balanceDiff = 0;
    if (closeAllRooms) {
      let totalGroupDebt = 0;
      for (const reservation of activeReservations) {
        const chargesTotal = await storage.getChargesTotal(reservation.id);
        const paymentsTotal = await storage.getPaymentsTotal(reservation.id);
        const roomTotal = parseFloat(reservation.totalRoomAmount || "0");
        totalGroupDebt += roomTotal + chargesTotal - paymentsTotal;
      }
    }
  } catch (error) {
    return res.status(500).json({ error: "Internal server error" });
  }
});

```

```

    }
    balanceDiff = totalAmount - totalGroupDebt; // positivo = paga de más, nega
  }

// Distribuir el pago entre las reservas activas
if (distribution === "equal") {
  const perRoom = totalAmount / activeReservations.length;
  for (const reservation of activeReservations) {
    await storage.createPayment({
      reservationId: reservation.id,
      amount: perRoom.toFixed(2),
      method,
      reference: reference || `Pago grupal - ${group.name}${closeAllRooms ? "
      date: new Date().toISOString().split("T")[0],
    });
  }
} else if (distribution === "proportional") {
  // Si closeAllRooms: distribuir exactamente el saldo pendiente de cada hab
  // para que queden todas en 0, independientemente del monto ingresado
  let totalCost = 0;
  const costs: { id: string; cost: number; balance: number }[] = [];
  for (const reservation of activeReservations) {
    const chargesTotal = await storage.getChargesTotal(reservation.id);
    const paymentsTotal = await storage.getPaymentsTotal(reservation.id);
    const roomTotal = parseFloat(reservation.totalRoomAmount || "0");
    const balance = roomTotal + chargesTotal - paymentsTotal;
    const cost = roomTotal + chargesTotal;
    costs.push({ id: reservation.id, cost, balance });
    totalCost += cost;
  }

  for (const item of costs) {
    let paymentAmt: number;
    if (closeAllRooms) {
      // Pagar exactamente el saldo pendiente de cada habitación
      paymentAmt = Math.max(0, item.balance);
    } else {
      const proportion = totalCost > 0 ? item.cost / totalCost : 1 / costs.length;
      paymentAmt = totalAmount * proportion;
    }
    if (paymentAmt > 0.001) {
      await storage.createPayment({
        reservationId: item.id,
        amount: paymentAmt.toFixed(2),
        method,
        reference: reference || `Pago grupal - ${group.name}${closeAllRooms ? "
        date: new Date().toISOString().split("T")[0],
      });
    }
  }
}

```

```

    });
  }
}

// Si closeAllRooms: hacer check-out de todas las reservas checked_in
let checkoutCount = 0;
if (closeAllRooms) {
  const today = new Date().toLocaleDateString("en-CA", {
    timeZone: "America/Argentina/Buenos_Aires"
  });

  for (const reservation of activeReservations) {
    if (reservation.status !== "checked_in") continue;

    // Registrar en changelog
    await db.insert(reservationChangelog).values({
      reservationId: reservation.id,
      operador: (req as any).user?.username || "sistema",
      tipo: "checkout_grupal",
      descripcion: `Check-out grupal con pago centralizado. Grupo: ${group.name}`
    });

    await storage.updateReservation(reservation.id, { status: "checked_out" });
    await storage.updateRoom(reservation.roomId, { status: "dirty" });

    // Crear tarea de housekeeping
    try {
      await db.insert(housekeepingTasks).values({
        id: randomUUID(),
        roomId: reservation.roomId,
        type: "checkout_clean",
        priority: "high",
        status: "pending",
        notes: `Check-out grupal (pago centralizado) - ${group.name}`,
        createdAt: new Date(),
      } as any);
    } catch {}

    checkoutCount++;
  }

  // Si todas las reservas están cerradas, marcar el grupo como finished
  const allDone = group.reservations.every(
    r => r.status === "checked_out" || r.status === "cancelled" ||
      activeReservations.some(ar => ar.id === r.id) // las que acabamos de
  );
}

```



```

    if (allDone) {
      await storage.updateGroup(req.params.groupId, { status: "finished" as any
    }
  }

  res.json({
    success: true,
    distributed: activeReservations.length,
    checkoutCount,
    balanceDiff: closeAllRooms ? balanceDiff : undefined,
  });

} catch (error) {
  console.error("Error processing group payment:", error);
  res.status(500).json({ error: "Error processing group payment" });
}
});

```

Cambio 3 — Importar `reservationChangelog` y `housekeepingTasks` en el scope del endpoint

Verificar que en el archivo `server/routes.ts` estén importados:

```

import { reservationChangelog, housekeepingTasks } from "@shared/schema";
import { randomUUID } from "crypto";

```

Si ya están importados en otros endpoints del mismo archivo, no duplicar.

NOTAS TÉCNICAS

- **Fix 1 es el más urgente.** El bug de edición borra datos porque `roomTypeId` queda `""` en el `formData`, y el `PATCH` lo sobrescribe en DB. El **Fix 3 (backend)** es la red de seguridad definitiva.
- En el **Fix 2**, cuando `distribution === "proportional"` y `closeAllRooms === true`, el sistema ignora el `amount` ingresado y paga exactamente el saldo de cada habitación. El `balanceDiff` informa al usuario la diferencia.
- El changelog de check-out grupal usa `tipo: "checkout_grupal"` — nuevo tipo, compatible con la tabla existente `reservation_changelog`.
- `storage.getChargesTotal` y `storage.getPaymentsTotal` ya existen y se usan en el

endpoint de check-out individual — reutilizar.

- Al limpiar el dialog de pago grupal en `onSuccess` , resetear también `setGroupPaymentCloseAll(false)` .
- El `Checkbox` viene de `@/components/ui/checkbox` (shadcn/ui) — ya disponible en el proyecto.